

Parallax: Performance Prediction for Training–Inference Co-Execution

ZESSEN HU, Harbin Institute of Technology, Shenzhen, Shenzhen, China

DESHENG WANG, Harbin Institute of Technology, Shenzhen, Shenzhen, China

SICHAO CHEN, Harbin Institute of Technology, Harbin, China

QIANG WANG, Harbin Institute of Technology, Shenzhen, Shenzhen, China

SHUO SI, Harbin Institute of Technology, Harbin, China

WEIZHE ZHANG, Harbin Institute of Technology, Shenzhen, Shenzhen, China

GPU clusters for deep learning (DL) workloads, especially inference, generally suffer from low utilization due to resource over-provisioning for satisfying their strict latency requirements. Co-locating throughput-oriented training jobs with latency-sensitive inference services is a promising approach to reclaim idle resources. However, accurately predicting the performance interference of such heterogeneous workloads under two mainstream GPU sharing mechanisms—Time-Slicing and Multi-Process Service (MPS)—remains a critical challenge. Existing predictors either focus on single-tenant scenarios or lack the fidelity to capture the complex contention patterns between training and inference. In this article, we present Parallax, a fine-grained and accurate performance prediction framework tailored for training–inference co-execution. Parallax introduces interpretable modeling strategies for the two primary GPU sharing paradigms. For Time-Slicing, we propose a simulation-based model leveraging the virtual time slice and switching overhead abstractions to reconstruct operator-level interleaving and context-switching costs. For MPS, we develop a two-stage framework that first predicts resource utilization under concurrency and then quantifies performance degradation caused by microarchitectural contention, such as memory bandwidth and cache. This resource-centric approach ensures robust generalization to unseen workload combinations. Extensive evaluations on modern GPUs demonstrate the high accuracy of Parallax, predicting execution latency with a MAPE of 4.33% for Time-Slicing and 6.12% for MPS across diverse DL models (averaged over training and inference). Parallax is available at <https://github.com/HIT-CeeCG/Parallax>.

New Paper, Not an Extension of a Conference Paper.

This work was supported by the National Natural Science Foundation of China (Grant No. 62402141), the Shenzhen Colleges and Universities Stable Support Program No. GXWD20231129102636001, the Guangdong Basic and Applied Basic Research Foundation (2025A1515011785), the open research fund of Pengcheng Laboratory (No.2025KF1B0020), the Guangdong Basic and Applied Basic Research Foundation (2023A1515110271), the Shenzhen Colleges and Universities Scientific Research Startup Foundation for Newly Introducing Advanced Scholars (entitled Research on Hybrid Deployment for Deep Learning Jobs in Intelligent Clusters), the National Natural Science Foundation of China (NSFC) / Research Grants Council (RGC) Collaborative Research Scheme (Grant No. 62461160332 & CRS_HKUST602/24), and 2025 Research and Development of Training-Inference Collaborative Management System Based on Domestic Computing Power Resource Pooling project from China Unicom Intelligent Manufacturing Technology and Industry (Guangdong) Co., Ltd. (No. YGS25AE1000002).

Authors' Contact Information: Zesen Hu, Harbin Institute of Technology, Shenzhen, Shenzhen, Guangdong, China; e-mail: huzesen@stu.hit.edu.cn; Desheng Wang (corresponding author), Harbin Institute of Technology, Shenzhen, Shenzhen, Guangdong, China; e-mail: wangdesheng@hit.edu.cn; Sichao Chen, Harbin Institute of Technology, Harbin, China; e-mail: chensichao@stu.hit.edu.cn; Qiang Wang, Harbin Institute of Technology, Shenzhen, Shenzhen, China; e-mail: qiang.wang@hit.edu.cn; Shuo Si, Harbin Institute of Technology, Harbin, China; e-mail: sishuo@stu.hit.edu.cn; Weizhe Zhang (corresponding author), Harbin Institute of Technology, Shenzhen, Shenzhen, Guangdong, China; e-mail: wzzhang@hit.edu.cn.



This work is licensed under a [Creative Commons Attribution 4.0 International License](https://creativecommons.org/licenses/by/4.0/).

© 2026 Copyright held by the owner/author(s).

ACM 1544-3973/2026/07-ART94

<https://doi.org/10.1145/3827621>

CCS Concepts: • **Computer systems organization** → **Single instruction, multiple data**; • **Software and its engineering** → **Scheduling**; **Software performance**; • **Computing methodologies** → *Model development and analysis*;

Additional Key Words and Phrases: GPU, performance prediction, training and inference, co-execution, interference

ACM Reference Format:

Zesen Hu, Desheng Wang, Sichao Chen, Qiang Wang, Shuo Si, and Weizhe Zhang. 2026. Parallax: Performance Prediction for Training–Inference Co-Execution. *ACM Trans. Arch. Code Optim.* 23, 3, Article 94 (July 2026), 25 pages. <https://doi.org/10.1145/3827621>

1 Introduction

Deep learning (DL) has achieved breakthrough advances and is now widely used in industrial production and scientific research. However, as model architectures grow increasingly complex and parameter counts keep expanding, the computational demand of **deep neural networks (DNNs)** has risen sharply, and conventional CPU-based processing can no longer satisfy real-world timeliness requirements. To improve the overall efficiency of DL workloads, high-performance devices—especially GPUs—are being deployed ever more broadly [16, 18, 22, 38, 43].

To provide robust and stable DL services with strict **Service Level Objectives (SLOs)**, major companies, such as Alibaba [1] and Google [12], typically deploy dedicated GPU clusters that reserve substantial headroom to absorb peak traffic [20, 24, 26]. As a result, their GPU utilization often remains persistently low, leaving a large fraction of GPU resources idle [37]. To more fully utilize this wasted capacity without violating inference SLOs, recent work has begun to explore co-running training and inference workloads on the same GPUs in inference clusters [6, 27, 40, 41]. The key idea is to opportunistically schedule training jobs alongside inference requests to increase hardware utilization. In practice, NVIDIA GPUs provide multiple mechanisms to share a physical device across tenants, including (i) driver-managed time-slicing (temporal multiplexing across processes), (ii) CUDA **Multi-Process Service (MPS)**, which enables concurrent kernel execution within a shared GPU context, and (iii) **Multi-Instance GPU (MIG)**, which partitions a GPU into isolated GPU instances. In this work, we focus on time-slicing and MPS because they are dynamic sharing mechanisms that exhibit non-trivial, runtime interference and thus require explicit modeling. In contrast, MIG relies on explicit instance (re)configuration, which is geared toward static provisioning rather than fine-grained runtime adaptation to short-term traffic; frequent changes can fragment resources and forfeit colocation synergy.

Recent interference-aware GPU multiplexing and model-serving systems typically build on or compare against these native sharing modes [29, 42, 44]. However, when training and inference share a single GPU, they inevitably compete for compute and memory resources in heterogeneous ways, leading to complex performance interference [7, 46]. This raises a critical challenge: when training and inference workloads execute concurrently on the same GPU, can the interference they induce still allow inference requests to satisfy their predefined SLOs?

Existing research (Table 1) does not yet provide a high-fidelity performance oracle for training–inference co-execution under native GPU sharing mechanisms. Prior work on GPU time-slicing mainly characterizes interference empirically or proposes custom schedulers, rather than predicting the latency of co-located deep learning workloads under default time-slicing [10, 15]. Under MPS, Mudi [6] models training–inference co-location via piece-wise linear fitting over resource partitions, largely focusing on inference SLO control with training treated as background. Meanwhile, gpulets/iGniter/Inss target inference-only multi-tenancy [8, 14, 45], and single-tenant predictors such as NeuSight, Habitat, and Li et al. [11, 23, 25] do not account for co-execution interference or

Table 1. Prediction Frameworks from Related Work

Framework	Year	Model	T/I Scenario	Train Err	Infer Err
Li et al. [25]	2023	Data-driven + Linear regression	I	/	7.00%
gpulets [8]	2022	Linear Regression	2I	/	10.26%
Moolchandani [28]	2022	ML model	nI	/	9.05%
iGniter [45]	2022	Analytical model	nI	/	3.90%
Inss [14]	2024	Analytical model	nI	/	1.59%
habitat [11]	2021	Runtime-based + ML model	T	11.80%	/
NeuSight [23]	2025	Analytical model + ML model	T/I	7.30%	9.70%
Mudi [6]	2025	Piece-wise linear function	3T1I	/	16.00%
Parallax (Ours)	2026	Analytical model + ML model	T/I/mTnI	4.72%	5.73%

Notes. T/I denotes a single training or inference task, and mTnI denotes m training tasks and n inference tasks executed in parallel. Train and Infer errors are taken from the corresponding original papers.

mechanism-specific effects. These gaps motivate Parallax: given a concrete time-slicing or MPS configuration, we aim at accurately predicting how training–inference co-location impacts both parties’ latencies.

To address this question, this article proposes a performance prediction framework for training–inference co-execution under native GPU sharing mechanisms. We develop interference models specialized for the two concurrency paradigms—time-slicing and MPS—yielding differentiated prediction paths that capture key cross-workload and cross-mode behaviors.

Specifically, the key contributions of this article are as follows:

- **Unified co-execution prediction framework:** We design and implement Parallax, a fine-grained and accurate performance prediction framework for training–inference co-execution. For the first time, the framework provides tailored, differentiated models for the two core GPU concurrency paradigms, time-slicing and MPS, thereby capturing complex resource contention and interaction effects with higher fidelity.
- **Time-Slicing Prediction Model:** We propose a **Time-Slicing Prediction Model (TSPM)** that accurately predicts co-executing training–inference workloads performance under GPU time-slicing. To address simulation distortion caused by the lack of publicly documented preemptive scheduling on GPUs, TSPM introduces two quantified abstractions: Virtual Time Slice and Switching Overhead. These abstractions enable high-fidelity emulation of interleaved operator-level training and inference, capturing real switching costs.
- **MPS performance prediction framework:** We develop a two-stage and interference-aware framework that uses hardware resources as the modeling intermediate, decomposing the problem into (1) predicting per-workload resource utilization under concurrency, and (2) quantifying performance interference induced by resource contention. By modeling at the root cause, competition for hardware resources, this approach mitigates the non-linearity that plagues direct performance modeling and achieves high accuracy and strong generalization even for previously unseen combinations of training and inference workloads.

We evaluate Parallax on modern GPU platforms using diverse benchmarks to assess prediction accuracy under both time-slicing and MPS concurrency. For time-slicing, through systematic calibration, Parallax precisely identifies the core parameters of TSPM. The predictions closely align with ground-truth measurements, achieving a MAPE of 4.33% across the dataset and thereby demonstrating robust generalization capabilities. For MPS, Parallax likewise performs strongly on both low-level GPU metrics and end-to-end performance prediction for concurrent workloads. On a more challenging test suite with complex, previously unseen scenarios (unseen-model and two-training), Parallax achieves a mean prediction error of 12.06% for low-level GPU metrics, and

attains a MAPE of 6.12% for train/inference workloads, further confirming its robustness under realistic co-execution settings.

2 Background and Motivation

2.1 GPU Architecture

Deep learning workloads expose rich fine-grained parallelism, and contemporary GPUs are architected precisely to harvest this parallelism when running many threads in parallel. Conceptually, a GPU consists of multiple identical **streaming multiprocessors (SMs)** that issue and schedule warps, all connected through an on-chip interconnect to a shared L2 cache and, beyond that, to off-chip device memory. Each SM integrates a collection of arithmetic pipelines and a small private cache (plus scratchpad/shared memory), enabling thousands of threads to execute in flight and hide long memory latencies. This regular, replicated design scales to tens of thousands of hardware threads and makes GPUs especially suitable for deep learning kernels—while also implying that performance prediction must carefully account for SM-level parallelism and the behavior of the memory hierarchy under parallel DL workloads.

2.2 Deep Learning and Training–Inference Co-Execution

Training and inference exhibit distinct resource and iterative characteristics, fundamentally motivating the design of co-execution architectures. The purpose of training is to extract features from large-scale datasets and optimize the model parameters. A typical training job executes thousands of iterations, each consisting of forward propagation, backward propagation, and parameter updates. Due to their iterative and compute-intensive nature, training jobs typically necessitate prolonged GPU occupancy to maintain efficient parallelism and gradient optimization. In contrast, inference uses a pre-trained model to generate predictions for user inputs and is often deployed as an online service that is invoked frequently. Unlike offline training, which can run for days or weeks, inference must handle unpredictable, bursty, and highly variable request patterns while meeting stringent sub-second latency targets [13, 39]. Since inference typically involves a single forward pass with low GPU demand, insufficient request rates often leave substantial GPU capacity underutilized [9, 47, 48].

Leading industry players like Alibaba [1] and Google [12] deploy multi-tenant, hyperscale GPU clusters to enhance inference throughput and efficiency. Unlike training, inference workloads are less resource-intensive, often failing to saturate high-end GPUs and prompting the deployment of lower-end alternatives like the NVIDIA T4. However, strict SLOs and traffic bursts require significant resource headroom. Consequently, inference clusters often exhibit sustained utilization below 40% [24], leaving substantial capacity idle.

To reclaim wasted capacity without violating inference SLOs, recent works have explored co-locating training and inference workloads [2, 27]. Figure 1 illustrates the motivation behind this approach. State (a) depicts an exclusive Inference Task A. State (b) reflects the production status quo: even with concurrent Inference Task B, significant GPU resources remain idle. This underutilization stems from two factors: first, clusters are over-provisioned to handle potential traffic bursts; second, inference demand is exogenous and fluctuates with diurnal user activity. During off-peak periods, request scarcity renders the surplus capacity unclaimable by inference workloads alone. To address this, State (c) introduces a Training Task C to scavenge these slack resources, thereby maximizing cluster efficiency.

2.3 Resource Multiplexing

To enhance GPU utilization, NVIDIA provides time-slicing and three spatial-sharing mechanisms.

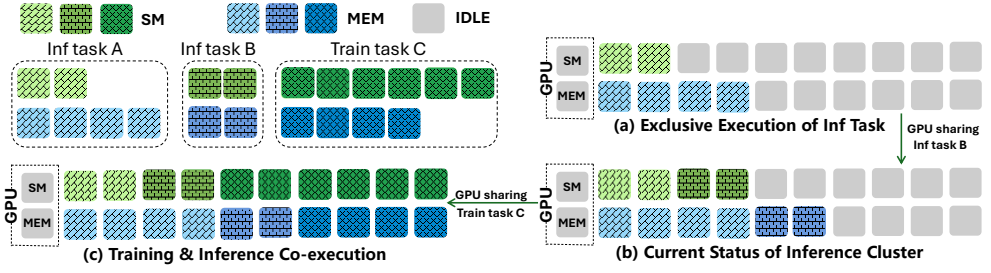


Fig. 1. Illustration of GPU resource utilization states.

Time-Slicing. As the default concurrency mechanism, time-slicing schedules independent processes via round-robin context switching. Strictly speaking, this is time-division multiplexing rather than true parallelism; GPU resources are assigned exclusively to a single process within each time slice [4, 35]. Although this introduces context-switching overhead and jitter, its support for logical replicas in Kubernetes [34] ensures widespread adoption in production environments.

Spatial Sharing. NVIDIA offers three approaches for concurrent execution: (1) **MPS** [30] logically partitions compute resources (e.g., SMs) among processes. However, the memory hierarchy (L2 cache, DRAM) remains shared, leading to potential interference. (2) **MIG** [31] provides strong isolation by statically partitioning a GPU into multiple instances, but its partitioning must be created/destroyed explicitly and is not designed for fine-grained, per-request runtime adaptation. This can lead to fragmentation and missed co-run opportunities under bursty inference. (3) **Priority Streams** allow kernel prioritization within a single process. However, they lack preemption capabilities for running blocks and necessitate intrusive code modifications to merge distinct workloads into a single process.

Summary. Given MIG’s static partitioning and typically offline configuration, and the implementation complexity of priority streams, our work focuses on modeling time-slicing and MPS, which are widely deployed for fine-grained GPU sharing in production. Nonetheless, MIG compatibility is discussed in Section 9.

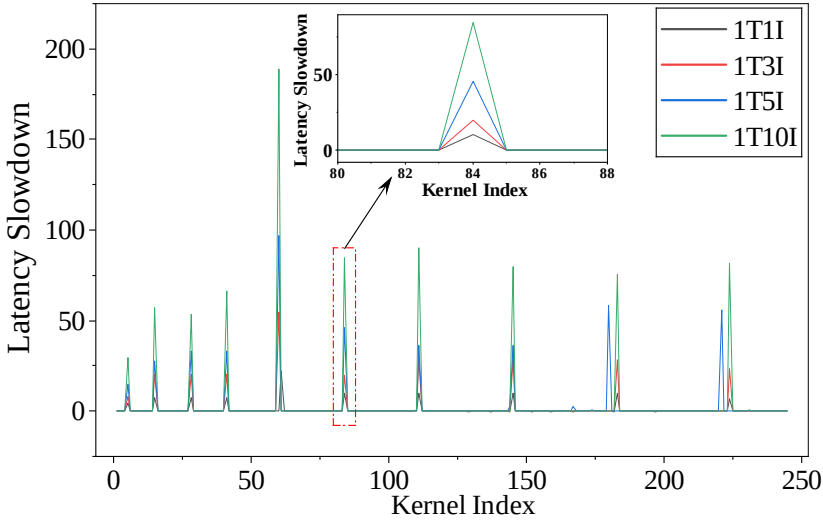
3 Understanding the Impact of Co-Execution

3.1 Characterizing Time-Sliced Co-Execution on NVIDIA GPUs

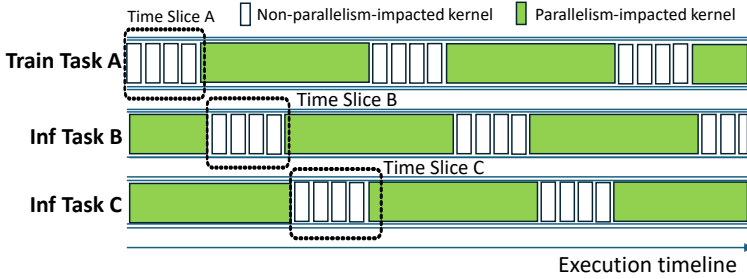
To characterize training–inference workload co-execution under the default time-slicing scheduling, we conduct a series of targeted experiments and distill three key observations (O1–O3). These observations not only elucidate the intrinsic regularities of time-slicing scheduling but also yield the core parameter abstractions for subsequent performance modeling—namely, Time Slice Size and Time Slice Switching Overhead.

O1: Deterministic Kernel Submission. A DL workload submits a fixed sequence of kernels to the GPU, irrespective of runtime concurrency or resource contention. This determinism is guaranteed by the static operator dependencies within the computational graph [3, 36]. Consequently, while co-location may alter the *timing* of kernel execution due to interference, it does not alter the *workload composition* itself. This insight simplifies our problem space, enabling us to treat the kernel stream as a constant input for performance prediction.

O2: Only a small subset of kernels exhibits concurrency-sensitive latency. Using ResNet-50 training and inference workloads, we collect kernel traces under five concurrency settings—1T0I, 1T1I, 1T3I, 1T5I, and 1T10I (one training workload co-executed with 1/3/5/10 inference workloads). Comparing the 1T0I baseline against 1TnI as shown in Figure 2(a), we observe that



(a) O2 : The impact of the time-slicing paradigm on workload kernels is inherently selective.



(b) O3 : Time-Slicing-Based Execution Workflow.

Fig. 2. The observations of time-slicing scheduling.

only a minority of kernels experience pronounced latency fluctuations, while the vast majority are nearly unaffected. As concurrency increases, the latency inflation of these sensitive kernels grows further. This indicates that time-slicing affects workloads selectively rather than uniformly across kernels, revealing underlying scheduling regularities and resource-contention patterns.

To further validate O2 and pinpoint which kernels/operators dominate the concurrency-sensitive latency, we analyze ResNet-50 traces under 1T1I and treat each profiled operator/kernel duration as its wall-clock GPU completion latency. For each operator type, we use its median solo duration as the baseline and mark an invocation as *impacted* if its slowdown exceeds $2 \times (d_{\text{corun}}/d_{\text{solo}} > 2)$.

Table 2. Operator-Level Attribution Under Time-Sliced Co-Execution (ResNet-50, 1T1I), Ranked by Contribution to Total Inflation Time

Training (Top-5 by p_Δ)			Inference (Top-5 by p_Δ)		
Operator type	p_Δ (%)	p_{solo} (%)	Operator type	p_Δ (%)	p_{solo} (%)
ConvolutionBackward0	31.36	30.62	aten::_convolution	23.56	14.02
aten::add_	11.33	4.93	aten::add	16.90	15.48
CudnnBatchNormBackward0	9.70	14.18	aten::convolution	16.53	7.75
aten::cudnn_batch_norm_backward	9.53	2.78	aten::cudnn_convolution	12.21	10.15
ReluBackward0	6.30	6.11	aten::cudnn_batch_norm	11.36	7.43
Top-5 sum	68.22	58.63	Top-5 sum	80.55	54.83

An invocation is marked impacted if its slowdown exceeds $2\times$ compared to the solo median baseline of the same operator type. p_Δ : share of total inflation time $\sum(d_{\text{corun}} - d_{\text{solo}})$ over impacted invocations; p_{solo} : solo time share.

Table 3. Sequential Baseline vs. Time-Sliced Co-Execution for ResNet-50 (Median Over Repeats)

Mode	Train completion (ms)	Infer completion (ms)	Makespan (ms)
Sequential (1T0I + 0T1I)	129.81	40.14	169.95
Co-run (1T1I)	176.11	84.13	176.11
Slowdown / Increase	1.36 $\times$	2.10 $\times$	+6.16 (3.63%)

Training time is one iteration (Forward + Backward + Optimizer); inference time is one batch.

Although only a minority of invocations are impacted, the resulting inflation is highly concentrated: the Top-5 operator types in Table 2 already account for 68.22% of training inflation and 80.55% of inference inflation. This concentration becomes even stronger for Top-10 (training: 85.77%; inference: 100.00%), directly supporting O2 that time-slicing interference is highly selective rather than uniform across kernels.

O3: GPU time-slicing uses round-robin scheduling and incurs switching overhead. Leveraging NVIDIA Nsight Systems [33], we visualize the execution timeline of co-located workloads. As illustrated in Figure 2(b) (depicting one training and two inference tasks), the scheduler strictly serializes execution via time-division multiplexing: the GPU is exclusively allocated to the Training Task for a specific quantum before rotating to Inference Tasks B and C. While this mechanism guarantees fairness, the requisite context switches introduce non-negligible overheads, directly exacerbating per-task service latency.

In the sequential baseline, we execute 1T0I followed by 0T1I on the same GPU and report median times. Training takes $T_{\text{solo}} = 129.81$ ms and inference takes $I_{\text{solo}} = 40.14$ ms, giving $M_{\text{seq}} = 169.95$ ms.

Under time-sliced co-execution (1T1I) with round-robin service, completion times increase to $T_{\text{corun}} = 176.11$ ms (1.36 $\times$) and $I_{\text{corun}} = 84.13$ ms (2.10 $\times$), yielding $M_{\text{corun}} = \max(T_{\text{corun}}, I_{\text{corun}}) = 176.11$ ms, i.e., $\Delta = 6.16$ ms (3.63%) above M_{seq} (Table 3).

Thus, time-slicing is not “free”: while overlap keeps the makespan increase modest, individual completion times—especially inference—still inflate substantially. The gap is consistent with O3, as slice-level gaps and switching overhead accumulate across rotations.

Building on the observations above, we model Time Slice Size and Switching Overhead as the key parameters of our performance model. This abstraction quantifies their influence on overall runtime performance by capturing the effects of context switching and scheduling, thereby providing a theoretical foundation for designing efficient optimization strategies for the co-execution of training and inference workloads.

Table 4. GPU Profiling Metrics used in Our Analysis

Metric	Resource	Description
Compute_Warps	Compute	Number of active warps (queued or executing).
SM_Issue_Active	Compute	Fraction of cycles issuing at least one instruction.
Memory_Read_Bandwidth	Memory	Achieved device-memory read bandwidth.
Memory_Write_Bandwidth	Memory	Achieved device-memory write-back bandwidth.
L1_Hit_Rate	Cache	Fraction of accesses served by the L1 cache.
L2_Hit_Rate	Cache	Fraction of accesses served by the L2 cache.

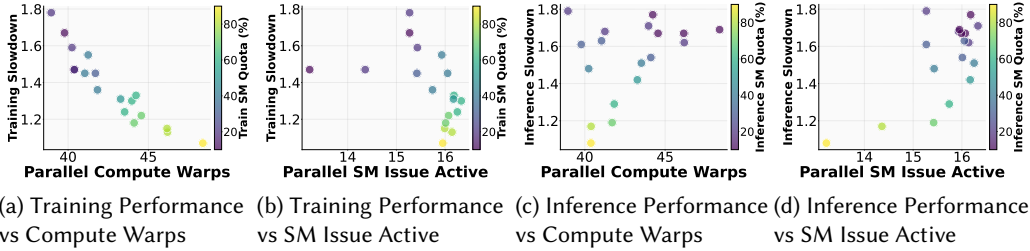


Fig. 3. Compute contention signatures vs. slowdown under MPS. Each point corresponds to one MPS configuration (SM quota allocation and concurrency setting) for co-running ResNet-50 training and inference. The x-axis reports the observed compute metric under that configuration, and the y-axis reports the measured slowdown. Color indicates the allocated SM quota.

3.2 Characterizing MPS-based Co-Execution on NVIDIA GPUs

Designed to mitigate context-switching costs, MPS enables spatial multiplexing by aggregating CUDA calls from multiple clients into a shared context [30, 49]. This architecture allows kernels from different processes to run simultaneously on SMs, significantly boosting utilization.

However, this design inherently sacrifices isolation. Although MPS supports logical SM partitioning, critical subsystems such as global memory bandwidth, L2 caches, and DRAM remain shared across all co-located processes [17]. Consequently, workloads compete for these finite resources, leading to complex performance interference. To address this, our predictive model focuses on quantifying contention in the two most critical domains: computation and memory.

3.2.1 Compute Resource Contention. GPU compute resources are primarily implemented by SMs, whose count is fixed on a given device. When multiple client processes submit kernels concurrently, they compete for SM cycles. The MPS server exposes the environment variable `CUDA_MPS_ACTIVE_THREAD_PERCENTAGE` to impose an upper limit on the share of compute resources of each client. However, this mechanism does not reserve dedicated SMs for a process—it merely enforces an upper bound. Kernels from different processes can still be scheduled in the same SMs, where the exact placement is determined by load-balancing policies. For performance analysis, we use `nsys` [33] to collect the compute-related metrics in Table 4, using `Compute_Warps` and `SM_Issue_Active` as proxies for compute utilization.

As illustrated in Figure 3, we co-run ResNet-50 training and inference under NVIDIA MPS and actively sweep the MPS SM quota allocations (in 10% steps) as well as the concurrency configuration. For each quota/configuration, we measure the end-to-end slowdown and collect GPU utilization/contention metrics via Nsight System. In Figure 3/4, the x-axis shows the observed metric value under a given quota/configuration, and the y-axis shows the corresponding

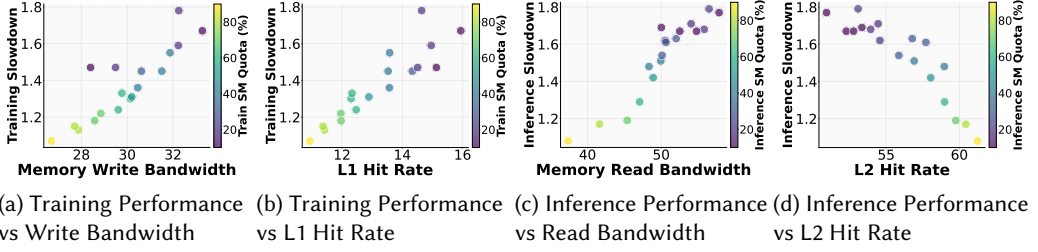


Fig. 4. Memory contention signatures vs. slowdown under MPS.

slowdown. This visualization highlights the strong association between resource contention signatures and slowdown, motivating our use of these metrics as predictive features in Section 6.

Our key observations in Figure 3 are as follows: (1) Across all four panels, there exists a strong correlation between the SM quota allocated to a workload and its corresponding slowdown. (2) Figure 3(a) clearly shows a pronounced negative correlation between the number of active compute warps and training slowdown. (3) In the remaining panels, after excluding minor outliers, strong correlations are also observed. (4) Higher overall compute utilization correlates with greater inference slowdown, which can likely be attributed to the fact that utilization is predominantly driven by the training workload. As a result, the inference workload receives a smaller effective SM quota and tends to experience an increased slowdown.

In general, computing resource utilization is strongly coupled with workload slowdown, and thus serves as an informative predictor of performance interference.

3.2.2 Memory Resource Contention. GPU memory resources consist of device-memory bandwidth and L1/L2 caches. Under MPS, these resources are shared equally across all processes, making them highly susceptible to contention and therefore leading to significant performance interference [17]. In our performance analysis, memory bandwidth and cache hit rates serve as key indicators of data-access efficiency, reflecting throughput and locality, respectively. Using nsys, we collect the memory- and cache-related metrics in Table 4, namely `Memory_Read_Bandwidth`, `Memory_Write_Bandwidth`, `L1_Hit_Rate`, and `L2_Hit_Rate`.

As shown in Figure 4, memory resources also exhibit strong correlations with workload slowdown:

- (1) Figure 4(a) indicates that training slowdown increases with total memory write bandwidth, which is consistent with the large volume of writes produced during backpropagation and parameter updates.
- (2) Interestingly, Figure 4(b) reveals a positive correlation between training slowdown and the SM-private L1 cache hit rate. Although a high L1 hit rate usually indicates good data locality, in this context it instead reflects the intensive computation activity of training workloads that saturate SM resources; hence, higher L1 hit rates emerge as a byproduct of heavier computation rather than efficiency gains.
- (3) Figure 4(c) clearly demonstrates a significant positive correlation between inference slowdown and GPU memory read bandwidth. Specifically, as the GPU memory read bandwidth increases, inference slowdown rises sharply, indicating that when co-running training workloads generate heavy memory traffic, intense contention for shared DRAM bandwidth becomes the primary bottleneck for inference, shifting it from compute-bound to memory-bound.

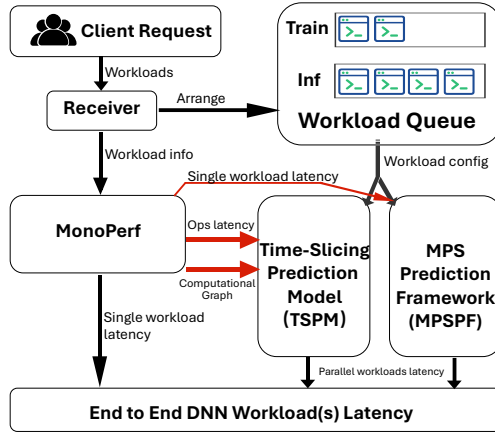


Fig. 5. Overall architecture of parallax.

- (4) As corroborating evidence, Figure 4(d) shows a strong negative correlation between the inference slowdown and the aggregate L2 cache hit rate. Higher L2 hit rates effectively mitigate DRAM pressure, thus significantly improving inference performance.

These findings underscore the asymmetric sensitivity of workloads to GPU memory resources: inference is predominantly bound by DRAM read bandwidth and L2 capacity, whereas training—characterized by intensive data reuse—relies on write bandwidth and L1 utilization. Consequently, performance degradation is deterministic rather than stochastic, and can be systematically predicted via hardware performance counters. These insights fundamentally motivate the need for bandwidth isolation and cache-aware scheduling to mitigate interference in co-located scenarios.

4 Design Overview of Parallax

4.1 Architecture and Workflow

Recognizing the distinct interference mechanisms of Time-Slicing and MPS, we present Parallax, a fine-grained and accurate performance prediction framework for training–inference co-execution. Parallax employs specialized prediction strategies for each paradigm. Figure 5 illustrates the overall architecture, comprising four key components that synergize to deliver high-fidelity predictions.

- **Receiver:** The Receiver serves as the entry point, maintaining separate queues for incoming training and inference tasks and managing their metadata for downstream processing.
- **MonoPerf:** Built by extending NeuSight [23], MonoPerf is a robust single-workload predictor that supports both Transformer and CNN architectures. It traverses the computational graph of the model to estimate operator-level latencies under isolated execution conditions. MonoPerf outputs three critical artifacts: (1) the computational graph structure, (2) per-operator latency profiles, and (3) the baseline end-to-end latency without interference.
- **Time-Slicing Prediction Model (TSPM):** Taking the graph and operator profiles from MonoPerf as input, TSPM simulates the time-sliced execution flow. By introducing two novel abstractions—*Virtual Time Slice* and *Switching Overhead*—it faithfully reproduces the operator-level interleaving and context-switching costs inherent to time-slicing, enabling precise latency forecasting.

- **MPS Prediction Framework (MPSPF)**: Designed for spatial sharing, MPSPF is a resource-centric, two-stage framework. It first predicts the resource utilization of concurrent workloads and then quantifies the performance degradation caused by hardware contention (e.g., memory bandwidth). Using the baseline latency from MonoPerf, it derives the final end-to-end latencies for co-located training and inference jobs.

4.2 Formulation of Performance Prediction

We consider performance prediction for an $mTnI$ configuration, denoting the co-execution of m training workloads and n inference workloads on a single GPU. Let $T = \{t_1, t_2, \dots, t_m\}$ and $I = \{i_1, i_2, \dots, i_n\}$ denote the queues of training and inference workloads, respectively. When a workload $w \in (T \cup I)$ arrives, the Receiver invokes MonoPerf using the descriptors of w and the target GPU configuration to predict its exclusive-run performance:

$$(CG_w, \mathbf{I}_w^{\text{ops}}, I_w^{\text{solo}}) = \text{MonoPerf}(C_w, \text{flops}, \text{mem}, \text{mem}_{\text{BW}}, L2_{\text{size}}), \quad (1)$$

where CG_w denotes the computational graph of the model, $\mathbf{I}_w^{\text{ops}}$ is the vector of per-operator latencies, and I_w^{solo} represents the end-to-end latency under exclusive execution. The configuration descriptor C_w includes the model name, batch size, workload type, and number of layers. Hardware parameters include the GPU's peak FLOP/s (flops), total device memory (mem), peak memory bandwidth (mem_{BW}) and L2-cache size ($L2_{\text{size}}$).

After predicting the single-tenant performance of a workload w , the upper-layer runtime (or scheduler) tags w based on its service semantics (e.g., throughput-oriented training vs. latency-sensitive inference, or more generally background vs. foreground) and places it into the corresponding logical queue (T or I). This tagging is provided by the system and is not inferred by the GPU/driver. Parallax does not assume that co-running workloads come from the same ML model; it only requires per-workload single-tenant profiles (operator graph/latencies and resource signatures) and a specified concurrency configuration.

$$(l_t, l_i, CG_{t,i}) = \text{TSPM}(CG_t, CG_i, \mathbf{I}_t^{\text{ops}}, \mathbf{I}_i^{\text{ops}}, S, O, m, n), \quad (2)$$

where l_t and l_i denote the predicted latencies of training workload t and inference workload i under co-execution, respectively. m and n are the numbers of co-running training and inference workloads. $CG_{t,i}$ represents the operator-level interleaving trace during co-execution, while S and O denote the time-slicing size and switching overhead.

$$(l_t, l_i, U_{t,i}) = \text{MPSPF}(I_t^{\text{solo}}, I_i^{\text{solo}}, C_i, C_t, R_i, R_t, m, n), \quad (3)$$

where $U_{t,i}$ denotes the predicted GPU utilization under co-execution, and R_i and R_t are the exclusive-run GPU utilizations of workloads i and t , respectively.

5 Time-Slicing Prediction Model

This section presents the Time-Slicing Prediction Model—a modeling framework for predicting the performance of co-located training–inference workloads under the GPU time-slicing execution paradigm. The model consists of two main components: (1) the simulation algorithm, and (2) the calibration of time-slicing size and switching overhead.

Time-slicing is the default sharing mode in many production GPU clusters, making accurate performance prediction crucial for SLO-aware colocation and admission control. Prior work on time-slicing primarily focuses on mechanisms and scheduling policies (e.g., faster context switching, iteration/request-level scheduling), but does not provide a general predictor for native GPU time-slicing under training–inference co-execution. A simplistic “uniform stretching” model fails

because time-slicing interference is highly non-uniform: most operators remain near-solo while a small subset is disproportionately inflated due to fragmentation across slices and context-switch gaps. Parallax addresses this gap by introducing a mechanistic simulator with two effective parameters—virtual slice size and switching overhead—and coupling it with operator-level profiles to accurately reconstruct co-execution timelines.

5.1 Simulation Algorithm

Motivated by the observations in Section 3.1, we premise our approach on NVIDIA’s default driver-managed time-slicing mechanism. While typically characterized as round-robin, the exact preemption granularity and bookkeeping details remain proprietary and opaque. Moreover, prior studies highlight that time-slicing induces non-negligible latency jitter due to context switching and resource contention [21].

To address this, rather than attempting to reverse-engineer these undocumented internals, we abstract the time-slicing mechanism using two effective parameters:

- **Virtual Time Slice (S)**: This denotes the expected GPU service quantum allocated to a workload per rotation. We designate it as “virtual” because actual execution is often fragmented by kernel boundaries; S aggregates these granular effects into a unified service budget.
- **Switching Overhead (O)**: This represents the aggregate non-productive time incurred during context handoffs, encompassing scheduler bookkeeping, state transitions, and microarchitectural perturbations.

Crucially, S and O serve as effective parameters summarizing the composite behavior of the driver scheduler, rather than static hardware constants.

Leveraging these abstractions, we propose the TSPM to characterize the scheduling dynamics of co-located training and inference workloads. As defined in Equation (2), the input to TSPM is a workload set $\mathcal{W} = \{t_1, \dots, t_m, i_1, \dots, i_n\}$, where each workload is profiled by its computational graph and operator-level baseline latencies. The simulation is driven by a global clock, `currentTime`, enforcing a two-level scheduling hierarchy: strict FIFO for operators within a process and Round-Robin across concurrent processes. In each scheduling rotation, TSPM grants a virtual time slice S to the active workload, simulates the execution of runnable operators until the budget is exhausted, accounts for the switching overhead O, and finally yields control. Algorithm 1 formalizes this procedure.

Algorithm 1 follows a two-level policy: FIFO within each workload and round-robin across workloads. In each rotation, the simulator grants the active workload a virtual slice S, executes ready operators until the slice budget is exhausted (preempting and later resuming an operator if needed), and then advances time by $S + O$ to account for the inter-slice switching gap. This directly mechanizes O2 (only operators that span slice boundaries are fragmented and inflate) and O3 (overhead accumulates across rotations).

To faithfully emulate execution, TSPM maintains, for each workload $w \in \mathcal{W}$, the following state information:

We illustrate Algorithm 1 with a minimal 1T1I co-execution instance. Assume a virtual time slice $S = 2.0$ ms and switching overhead $O = 0.1$ ms. The training workload has F1 (0.6 ms) $\rightarrow$ F2 (0.8 ms) $\rightarrow$ B1 (2.6 ms), and the inference workload has I1 (1.0 ms) $\rightarrow$ I2 (0.7 ms). In the first slice, training completes F1 and F2 and partially executes B1, which is then preempted at the slice boundary; inference completes within the second slice; training resumes and finishes B1 in the third slice, completing at 6.2 ms. Only the long operator B1 is fragmented across slices (O2), while the accumulated gaps across rotations are captured by O (O3).

- **Ready queues**: A forward-propagation queue (`fw_ready`) and a backward-propagation queue (`bw_ready`) that hold operators whose dependencies are satisfied.

ALGORITHM 1: Time-Slicing Simulation Algorithm

Input: workloads = $\{W_1, \dots, W_n\}$ with operator graphs and per-op base latency $\tau(\cdot)$;
S: Time-slicing size;
O: Switching overhead;
Output: CompletionTimes: map from workload $\rightarrow$ completion timestamp;

```

1  currentTime  $\leftarrow$  0;
2  Initialize ready queues and remainingTime for each workload;
3  while there exists a workload not completed do
4      foreach workload  $W_i$  in workloads do
5          sliceStart  $\leftarrow$  currentTime;
6          if  $W_i$  already completed then
7               $\quad$  currentTime  $\leftarrow$  sliceStart + S + O;
8          else
9              timeInSlice  $\leftarrow$  S;
              // prefer resuming an interrupted op; otherwise pick from fw_ready then
              bw_ready
10             while timeInSlice > 0 and  $W_i$  has a ready operator do
11                 op  $\leftarrow$  next ready operator for  $W_i$ ;
12                 exec  $\leftarrow$  min(remainingTime(op), timeInSlice);
13                 remainingTime(op)  $\leftarrow$  remainingTime(op) - exec;
14                 timeInSlice  $\leftarrow$  timeInSlice - exec;
15                 if remainingTime(op) = 0 then
16                     mark op complete; release dependents into ready queues;
17                     if all ops of  $W_i$  complete then
18                          $\quad$  CompletionTimes[ $W_i$ ]  $\leftarrow$  sliceStart + (S - timeInSlice);
19             currentTime  $\leftarrow$  sliceStart + S + O;
20 return CompletionTimes;
```

- **Current_{op}**: The operator left unfinished when the previous slice was preempted.
- **RemainingTime(v)**: The remaining execution time required by the operator v .

5.2 Calibration of Time-Slicing Size and Switching Overhead

Under default time-slicing, the degree of process concurrency governs the round-robin rotation interval, directly influencing the frequency of context switches and the probability of operator preemption. While vendor specifications remain opaque, prior empirical studies consistently suggest a fixed time quantum, typically on the order of 2 ms on modern architectures.

We calibrate the effective parameters (S, O) via grid search. Given a small calibration set with measured GPU service time, we sweep (S, O) over $[S_{min}, S_{max}] \times [O_{min}, O_{max}]$, run TSPM for each candidate pair, and select the pair that minimizes MAPE between predicted and measured service time (Equation (4)). The optimal parameters (S^*, O^*) are obtained by argmin over the grid (Equation (5)).

$$\text{MAPE}(S, O) = \frac{1}{M} \sum_{i=1}^M \left| \frac{T_{\text{actual}}^{(i)} - T_{\text{pred}}^{(i)}(S, O)}{T_{\text{actual}}^{(i)}} \right|. \quad (4)$$

$$(S^*, O^*) = \arg \min_{(S, O)} \text{MAPE}(S, O). \quad (5)$$

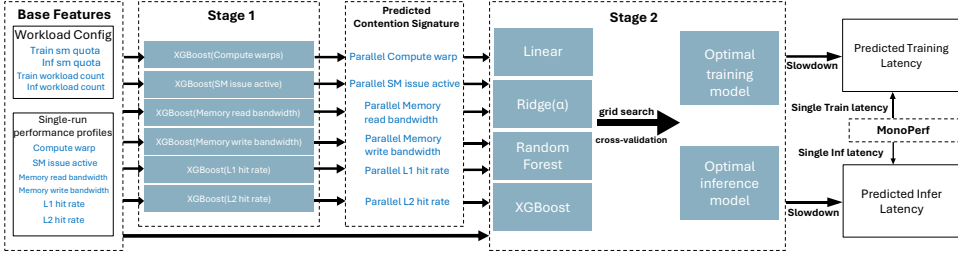


Fig. 6. Overall data flow of the MPS prediction framework.

6 MPS Prediction Framework

This section presents a resource-mediated, two-stage, interference-aware prediction framework that, under NVIDIA MPS, captures the structural relationship between compute/memory resource states and workload slowdowns during training–inference co-execution. The core idea is threefold: (1) predict resource utilization under co-execution, (2) estimate slowdown conditioned on that resource state, and (3) combine these with the single-workload latencies from MonoPerf to derive per-workload latencies for the co-executing training and inference workloads.

The overall data flow of the MPS Prediction Framework is shown in Figure 6.

In the first stage, we model the co-execution resource state R^{co} as a multi-output regression problem. For each metric $r \in R^{\text{co}}$, we train an individual regressor g_r :

$$R^{\text{co}} = \{\text{warps}, \text{sm}, \text{BW}_{\text{read}}, \text{BW}_{\text{write}}, \text{L1}_{\text{hit}}, \text{L2}_{\text{hit}}\}. \quad (6)$$

Here, *warps* denotes *Compute_Warps*, *sm* denotes *SM_Issue_Active*, BW_{read} denotes *Memory_Read_Bandwidth*, BW_{write} denotes *Memory_Write_Bandwidth*, L1_{hit} denotes *L1_Hit_Rate*, and L2_{hit} denotes *L2_Hit_Rate*.

6.1 Resource Utilization Prediction

We use a set of base features X_{base} as inputs, which depend only on static workload configurations and single-workload performance, and are fully known prior to deployment:

- **Workload configurations** C_t and C_i : including *train_sm_quota* and *inf_sm_quota*, encoding the compute-share allocations for training and inference, respectively.
- **Single-run performance profiles** R_t and R_i : hardware-counter-derived indicators that serve as the “signature” of each workload, reflecting its intrinsic resource demand.

For the regressors g_r , we use the open-source machine learning library, XGBoost [5], to learn the mapping from the base features to the co-execution resource state:

$$\hat{R}^{\text{co}} = \text{XGBoost}(X_{\text{base}}). \quad (7)$$

We choose XGBoost for its strong ability to model nonlinear relationships and its robustness to heterogeneous, mixed-scale features.

6.2 Workload Performance Prediction

After estimating the co-execution resource state $\hat{R}^{\text{co}}$, the second stage synthesizes this information to predict the slowdowns of training and inference workloads. The inputs at this stage are the union of the base features X_{base} and the first-stage estimate $\hat{R}^{\text{co}}$. This augmented feature set is central to the accuracy of the MPS Prediction Framework, as it combines static workload attributes with

Table 5. Concurrent Scenarios for the Time-Slicing Prediction Model

Dataset	Task Combo and Parallelism	Notes
Calibration	C1 & ResNet-50 train/inference	1T1I, 1T2I, 1T3I, 1T4I
	C2 & BERT-Large train/inference	1T1I, 1T2I, 1T3I, 1T4I
Validation	V1 & ResNet-101 train/inference	1T1I, 1T2I, 1T3I, 1T4I
	V2 & ResNet-152 train/inference	1T1I, 1T2I, 1T3I, 1T4I
	V3 & EfficientNet-b0 train/inference	1T1I, 1T2I, 1T3I, 1T4I
	V4 & Mobilenetv3-Large train/inference	1T1I, 1T2I, 1T3I, 1T4I
	V5 & ResNet-50(T) & ResNet-101(I)	1T1I, 1T2I, 1T3I, 1T4I
	V6 & GPT-2 Large train/inference	1T1I, 1T2I

an explicit characterization of dynamic, runtime resource contention, providing richer and more direct predictive signals.

To select the most suitable predictor for training–inference workloads, we consider a candidate model family:

$$\mathcal{F} = \{\text{Linear}, \text{Ridge}(\alpha), \text{RandomForest}, \text{XGBoost}\}, \quad (8)$$

and, through grid search and cross-validation, choose separate optimal models for training and inference slowdowns.

7 Performance Evaluation

7.1 Experimental Setup

Testbed. Our primary evaluation is conducted on a server equipped with dual Intel Xeon Platinum 8358P CPUs (128 cores), 503 GB DRAM, and two NVLink-interconnected NVIDIA RTX A6000 GPUs (48 GB). To further assess Parallax’s robustness across different hardware generations, we extend our validation to two additional platforms: (1) a high-end training node featuring NVIDIA A100 (40 GB) GPU, representing Ampere architecture; and (2) an inference-focused edge server equipped with NVIDIA T4 (16 GB) GPU, representing the Turing architecture. The software stack remains consistent across all platforms (Ubuntu 22.04, CUDA 12.9, PyTorch 2.5.1).

Profiling Tools. We leverage the NVIDIA Nsight suite for performance analysis. Specifically, we use Nsight Systems (*nsys*) [33] to capture system-level timelines, identifying kernel execution patterns, memory transfers, and scheduling behaviors (e.g., round-robin O3 patterns under time-slicing).

Workloads and Datasets. We curate distinct datasets to evaluate TSPM and MPSPF, covering diverse compute-memory intensities using five representative model families: ResNet-50/101/152, EfficientNet-b0, MobileNetv3-Large, BERT-Large, and GPT-2 Large.

(1) *Time-Slicing*: Workloads are divided into a **Calibration Set** (for parameter tuning) and a **Validation Set** (for generalization testing), as detailed in Table 5. We collect exclusive-run operator latencies and dependency graphs via MonoPerf.

(2) *MPS*: We construct diverse co-execution scenarios by varying the number of concurrent workloads: specifically, we deploy 1 to 2 training jobs alongside 1 to 4 inference instances. We systematically sweep the SM quota allocations for each combination in 10% increments (ensuring the total sums to 100%), covering a wide range of compute–memory intensity profiles. This yields 601 samples for model training and validation. To rigorously test generalization, we introduce three unseen models (BERT-Base, GPT-2 Medium/Small) and scenarios with two concurrent training jobs, generating a total of 1,266 Test Samples (1,002 unseen-model + 264 multi-training).

Latency Metric Selection. We focus on per-iteration GPU service time, defined as the isolated device-side execution latency for a training step or inference batch. This choice is strategic: accurate

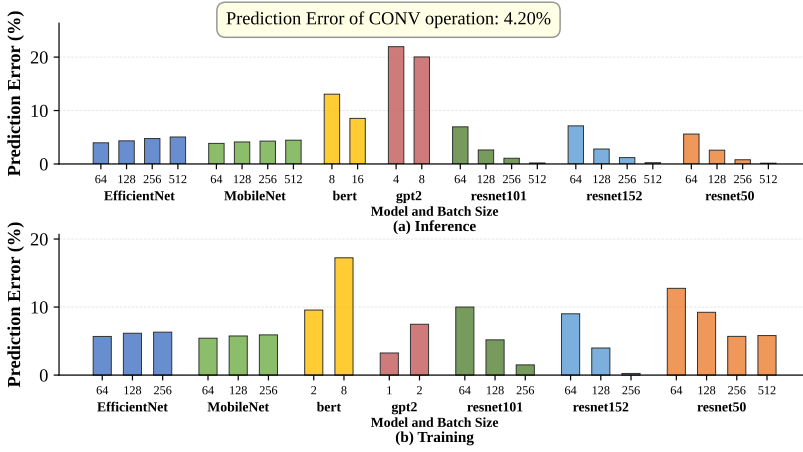


Fig. 7. Prediction accuracy of monoparf.

service time is the fundamental prerequisite for resource partitioning, while queuing-induced delays are orthogonal and derivable via standard queuing models. Consequently, we evaluate Parallax primarily using scale-free metrics (e.g., MAPE) and mean errors to characterize both accuracy and robustness.

Baselines. For MPS, we adapt **gputelets** [8] as the baseline. Originally designed for inference co-location, gputelets uses linear regression on offline-profiled L2/DRAM metrics to predict interference. We extend it to support training–inference scenarios. For TSPM, there is no direct baseline as prior works do not target time-sliced training–inference co-execution.

7.2 Validation of the Base Predictor

To enable operator-level latency profiling, Parallax requires a high-fidelity single-workload predictor. We adopt the methodology of NeuSight [23], a state-of-the-art predictor originally tailored for Transformer-based architectures. However, since our evaluation scope encompasses both NLP and Computer Vision workloads, NeuSight’s native support for CNNs was insufficient.

To bridge this gap, we extended NeuSight’s analytical logic to support convolution operators, implementing a module that we refer to as *MonoPerf*. We emphasize that MonoPerf is not a standalone contribution, but a necessary infrastructure component to ensure diverse workload coverage. Figure 7 validates its reliability: across diverse batch sizes and model architectures, MonoPerf attains an average prediction error of 4.20% for convolution operators, and aggregate errors of 2.61% for inference and 6.34% for training on ResNet variants. In addition, for MobileNetV3 and EfficientNet, MonoPerf achieves an average prediction error of 5.87% on training workloads and 4.35% on inference workloads. This high accuracy confirms that MonoPerf provides a solid baseline for the subsequent interference modeling in Parallax.

7.3 Performance of Time-Slicing Prediction Model

7.3.1 Calibration of Model Parameters (S, O). We calibrate the two effective time-slicing parameters in TSPM—the virtual slice size S and the switching overhead O —using a small calibration set and grid search that minimizes the MAPE between predicted and measured GPU service time. For CNN-style workloads (Dataset C1; ResNet-50) on RTX A6000, we sweep $S \in [1.5, 2.5]$ ms and $O \in [0.01, 0.1]$ ms; Figure 8 shows a well-behaved error landscape with a clear optimum, yielding $(S_{image}^*, O_{image}^*) = (2.40ms, 0.01ms)$ (MAPE 5.92%). CNN-derived parameters do not directly transfer

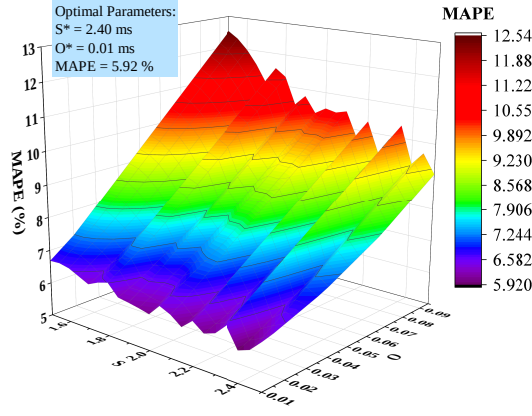
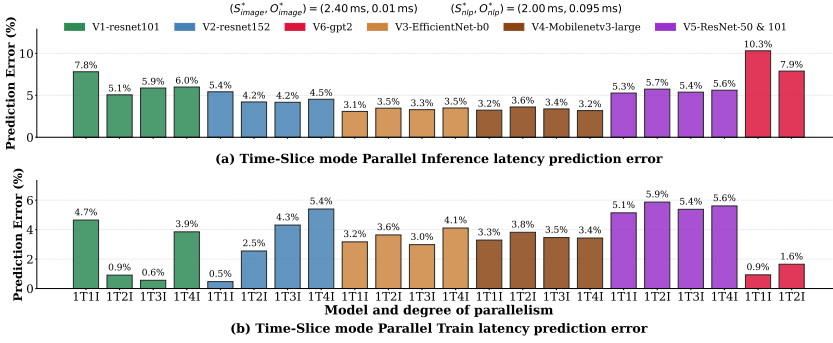
Fig. 8. C1's calibration of tspm parameters (S, O).

Fig. 9. Prediction errors of TSPM across diverse training–inference co-execution scenarios.

to Transformer-style workloads due to their fine-grained, bandwidth-bound operators, which amplify sensitivity to slice-level gaps; calibrating on NLP workloads (Dataset C2; BERT-Large) gives $(S_{nlp}^*, O_{nlp}^*) = (2.00ms, 0.095ms)$. In practice, the model family is known *a priori*, so Parallax applies the corresponding (S, O) profile based on the workload topology, avoiding per-model manual tuning while preserving accuracy across heterogeneous architectures.

7.3.2 Prediction Error of TSPM. To evaluate TSPM's generalization and final accuracy, after calibration, we evaluate on a held-out validation set (Table 5). Using the fixed, calibrated parameters $(S_{image}^*, O_{image}^*)$ and (S_{nlp}^*, O_{nlp}^*) , we predict the performance for validation scenarios and compare the predicted GPU service times with ground-truth measurements.

As shown in Figure 9, the predicted results closely match the measured values: the mean prediction error is 3.75% for training workloads, 4.91% for inference workloads, and the overall MAPE across all validation cases is 4.33%, demonstrating both high accuracy and strong generalization capability.

7.4 Performance of MPS Prediction Framework

This section evaluates the proposed MPS prediction framework along three dimensions: (1) prediction of GPU metrics, (2) selection of the second-stage regressor, and (3) overall prediction accuracy.

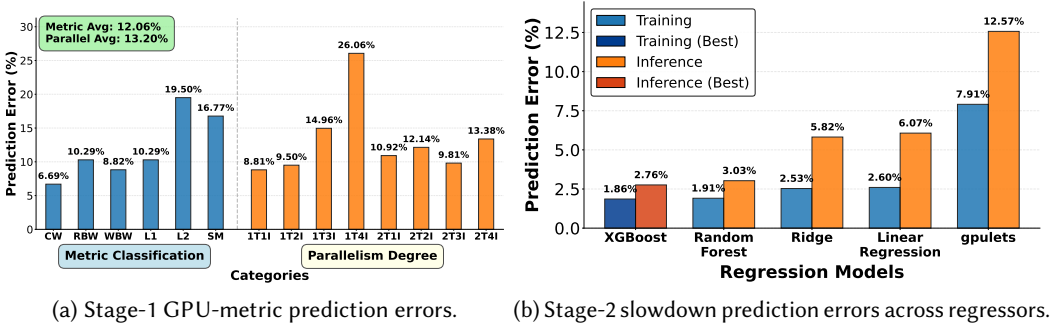


Fig. 10. Accuracy of the two-stage MPS slowdown predictor.

7.4.1 Prediction of GPU Metrics. For co-executing training–inference workloads, low-level GPU performance counters (e.g., Compute Warps, Memory Bandwidth, and Cache Hit Rates) are key determinants of interference. In Stage 1, we employ XGBoost to predict six concurrent metrics: *Compute Warps*, *Memory Read Bandwidth*, *Memory Write Bandwidth*, *L1 Hit Rate*, *L2 Hit Rate*, and *SM Issue Active*. The training and test sets contain 601, and 1266 samples, respectively.

As shown in Figure 10(a), XGBoost achieves accurate predictions for the six low-level GPU resources listed in Table 4 under various degrees of parallelism. Specifically, the model demonstrates exceptional precision on critical execution metrics, with prediction errors for **Compute Warps (CW)** and **Memory Write Bandwidth (WBW)** as low as 6.69% and 8.82%, respectively. Although high-variance metrics like L2 cache exhibit slightly higher deviations (19.50%), the overall average error across all metrics is contained at 12.06%. Furthermore, across different parallelism configurations, the model maintains a consistent average error of 13.20%, validating its robustness in capturing resource contention.

7.4.2 Selection of the Second-Stage Regressor. As summarized in Figure 10(b), we select XGBoost as the Stage-2 regressor for both training and inference, achieving the lowest mean error for slowdown prediction (1.86% and 2.76%, respectively). Although linear regression is the weakest among the evaluated models, it still substantially outperforms GPULETS (2.60% vs. 7.91% for training and 6.07% vs. 12.57% for inference), confirming that our Stage-1 metrics are highly informative for predicting MPS slowdown.

7.4.3 Prediction Error of the MPS Prediction Framework. To evaluate the generalization and accuracy of the MPS prediction framework, we test the best models identified in Section 7.4.2 on 1266 samples (1,002 model-unseen and 264 two-training). As shown in Figure 11, across a wide range of concurrency levels, the framework achieves prediction errors of 5.69% for training and 6.54% for inference. Aside from the most complex 2T4I configuration, which exhibits a relatively high prediction error of 14.46%, Parallax maintains a maximum prediction error of only 7.46% across the other configurations.

These results confirm that the MPS prediction framework can effectively predict the execution behavior of training and inference workloads under diverse degrees of parallelism. Its strong performance stems from explicitly modeling slowdown through GPU resource contention dynamics rather than relying solely on offline profiling statistics.

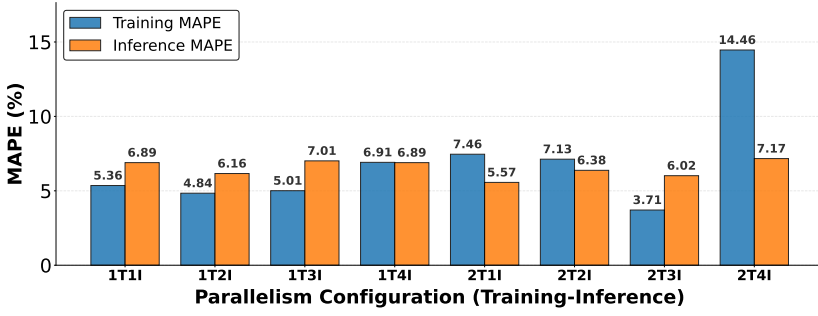


Fig. 11. Performance of the MPS prediction framework.

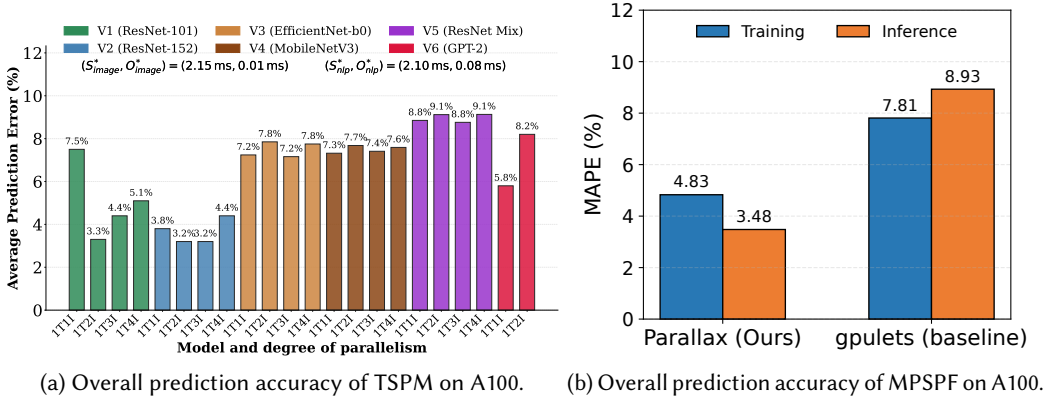


Fig. 12. The performance of Parallax on data center GPU (NVIDIA A100).

7.5 Cross-Architecture Scalability

To validate that PARALLAX generalizes beyond a specific GPU platform, we additionally evaluate it on two distinct NVIDIA GPUs: **A100** (40 GB, Ampere) and **Tesla T4** (16 GB, Turing). Compared to the A6000, these devices differ substantially in compute capability and memory/bandwidth characteristics, providing a stringent cross-platform test. We reuse the same calibration and data-collection protocol as Section 7.1 without modifying the model design.

7.5.1 Performance on Data Center GPU (NVIDIA A100). As shown in Figure 12, we first evaluate PARALLAX on the A100 to assess its scalability in high-performance computing scenarios. For the TSPM, the calibration process reveals that the optimal time-slicing parameters on A100 converge to $(S_{image}^*, O_{image}^*) = (2.15 \text{ ms}, 0.01 \text{ ms})$ and $(S_{nlp}^*, O_{nlp}^*) = (2.10 \text{ ms}, 0.08 \text{ ms})$. Using these device-specific parameters, TSPM achieves a MAPE of **6.58%** on the dataset. This result indicates that TSPM successfully adapts to the scheduling characteristics of high-end GPUs.

For the MPS Prediction Framework, we collected a new dataset of training–inference pairs on the A100. Despite the distinct memory characteristics of HBM2 compared to GDDR6, the selected performance counters (e.g., active warps, memory bandwidth utilization) remain effective indicators of workload interference. The framework maintains high accuracy with a mean prediction error of **4.83%** for training and **3.48%** for inference workloads, which are lower than gpulets (7.81% and

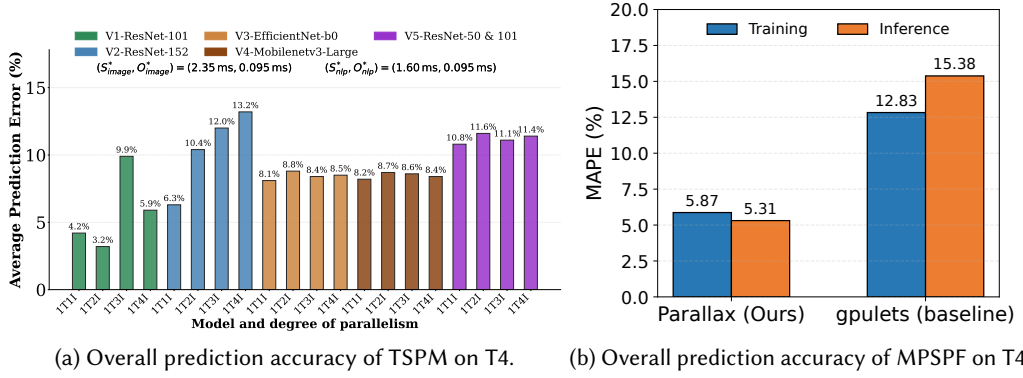


Fig. 13. The performance of Parallax on Inference GPU (NVIDIA T4).

8.93%). These results confirm that PARALLAX successfully scales to high-end accelerators, accurately modeling interference even under the high parallelism enabled by the A100.

7.5.2 Generalization to Inference GPU (NVIDIA T4). Next, as shown in Figure 13, we examine the robustness of PARALLAX on the NVIDIA T4, a representative platform for cost-effective inference. Unlike the Ampere-based A6000 and A100, the T4 utilizes the older Turing architecture and possesses significantly constrained memory bandwidth and compute resources. Such resource constraints typically exacerbate contention between co-located workloads, making performance prediction more challenging. Specifically, the limited memory capacity of the T4 precludes the training of gpt2-large; consequently, we exclude this model from the dataset for the T4 evaluation.

Through the calibration phase, we identified the parameters for TSPM as $(S_{image}^*, O_{image}^*) = (2.35 \text{ ms}, 0.095 \text{ ms})$ and $(S_{nlp}^*, O_{nlp}^*) = (1.60 \text{ ms}, 0.095 \text{ ms})$. With these parameters, TSPM yields a prediction error of **8.89%**, demonstrating that our time-slicing modeling approach possesses cross-generational applicability.

Regarding the MPSPF, Parallax demonstrates exceptional adaptability in this resource-constrained environment. After retraining with T4-specific data, the framework achieves prediction errors of **5.87%** and **5.31%** for training and inference, respectively. Crucially, compared to the baseline gpulets, PARALLAX reduces the prediction error by approximately **60.4%**. This significant improvement highlights the superiority of our explicit resource-contention modeling in scenarios where hardware resources are scarce and interference is severe.

7.6 Case Study

7.6.1 SLO-Aware Dynamic Batching and Admission Control under Time-Slicing. In production serving, inference batch size (micro-batch) is a critical tunable that trades throughput for latency. Given a running training job, how should a scheduler jointly choose the inference concurrency n and the maximum safe micro-batch size b^* to maximize throughput while respecting SLOs?

Using TSPM, we fix a training job (ResNet-101) and sweep inference (ResNet-152) micro-batch size under three concurrency settings (1T1I, 1T2I, 1T3I). Figure 14 shows the predicted inference service time $L_{pred}(I)$. Inference throughput is computed as $b/L_{pred}(I)$. While each curve is approximately linear, the slope steepens dramatically with higher n due to selective operator fragmentation (O2) and accumulated switching overhead (O3, see Table 2). For SLO=1000 ms, Parallax recommends

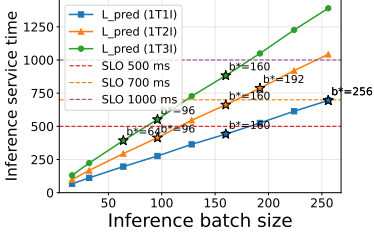


Fig. 14. Predicted inference service time $L_{\text{pred}}(I)$ vs. inference micro-batch size b under different co-execution settings (1T1I/1T2I/1T3I).

$b=160$ (1T1I), $b=96$ (1T2I), and $b=64$ (1T3I), respectively, enabling up to $2.1\times$ higher inference throughput than conservative $b^*=32$ while guaranteeing the SLO.

This case demonstrates that Parallax serves as a practical oracle for joint batching and admission decisions—precisely the capability required by modern SLO-aware systems.

7.6.2 MPS Quota Tuning Under Inference SLO. Under NVIDIA MPS, SM quota is a practical knob to control co-location interference. Given a fixed pairing ($T = \text{ResNet-101}$ training, $I = \text{BERT-large}$ inference) and a fixed concurrency setting (1T2I or 1T1I), we sweep train_sm_quota and use Parallax to predict the inference service time $L_{\text{pred}}(I)$ (blue) and training throughput (red) under each quota. We impose an absolute inference SLO $L_{\text{pred}}(I) \leq 1500$ ms (green line) and select the quota that maximizes training throughput within the SLO-safe region. Figure 15 shows that Parallax identifies a clear feasibility boundary and recommends $\text{train_sm_quota}=60\%$ for 1T2I and 80% for 1T1I (stars), achieving higher training throughput while keeping inference SLO-safe; relative to the 50/50 split, the recommended quotas improve training throughput by 63% (1T2I) and 81% (1T1I), respectively. This case study demonstrates that Parallax turns interference prediction into a deployable decision rule: maximize training throughput subject to the inference SLO constraint.

8 Related Work

We group prior work on deep learning performance prediction into three lines: (1) single-workload prediction under exclusive GPU execution, (2) concurrent multi-tenant inference modeling, and (3) training-inference co-location (Table 1).

Single-Workload Performance Prediction. Single-job predictors estimate latency/throughput when a model runs alone on a GPU. Habitat [11] and Li et al. [25] build learned predictors for standalone training or inference, while NeuSight [23] combines analytical and learned components to provide fine-grained per-operator estimates. In contrast, Parallax targets co-execution where heterogeneous training and inference share a GPU, and the dominant challenge becomes modeling interference from contention and GPU sharing mechanisms (Time-Slicing/MPS), which is out of scope for mono-workload predictors.

Concurrent Inference Performance Modeling. Several systems model interference among inference-only workloads in multi-tenant serving, such as gpulets [8], iGniter [45], and Inss [14].

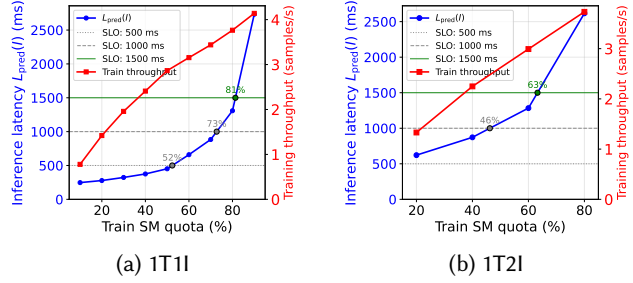


Fig. 15. Case study: Predicted inference service time vs. predicted training throughput as train_sm_quota varies. Horizontal lines denote inference SLOs; quota settings whose predicted latency lies below a given SLO line satisfy that SLO. The recommended quota is the feasible setting with the highest predicted training throughput. $T=\text{ResNet-101}$, $I=\text{BERT-large}$.

These works primarily address n -inference co-run scenarios and often rely on heavyweight kernel-counter sampling (e.g., Nsight Compute [32]) to parameterize interference models, which can be costly for online use. Parallax instead focuses on training-inference co-location and uses lower-overhead profiling (Nsight Systems [33]) together with mechanism-aware modeling to provide a high-fidelity oracle under native GPU sharing.

Training-Inference Co-Location. Mudi [6] studies training-inference co-location under MPS-style partitioning and fits inference latency as a piece-wise linear function of allocated shares, mainly treating training as background and focusing on inference control for SLO-driven placement. Parallax differs by (i) unifying prediction across both native sharing mechanisms (Time-Slicing and MPS), (ii) predicting performance impact for both training and inference, and (iii) explicitly modeling mechanism-level effects (operator interleaving and switching overhead for Time-Slicing; resource-mediated contention modeling for MPS). This broader scope and higher-fidelity modeling are necessary to support QoS-aware co-execution planning across diverse workloads and GPUs.

9 Discussion

The modular architecture of Parallax—specifically, the decoupling of single-workload profiling from mechanistic concurrency modeling—serves as a robust foundation for scaling beyond single-GPU scenarios. Below, we discuss how this design facilitates extensions to distributed systems, alongside its workload generalization and integration feasibility.

MIG compatibility. Although we do not present a full-fledged evaluation of MIG configurations, Parallax is compatible with MIG in a straightforward manner. When workloads are isolated in different MIG instances, interference is largely eliminated and Parallax reduces to single-tenant prediction per instance. When multiple workloads share one MIG instance using time-slicing or MPS, Parallax’s TSPM/MPSPF apply with minimal changes by using the MIG instance as the target device (with correspondingly scaled compute and memory resources).

Extensibility to Distributed Systems. Prior work models inter-GPU communication (e.g., all-reduce/all-gather) as graph operators and predicts end-to-end performance by jointly accounting for compute and communication [19, 23]. Since Parallax already operates on operator graphs, extending it to multi-GPU/multi-node settings mainly requires (i) adding communication operators/features with topology- and bandwidth-aware signals (e.g., NVLink/PCIe/RDMA, transfer volume) and (ii) extending the concurrency model to capture cross-device overlap and contention. This yields a natural path to support data/tensor/pipeline parallelism within the same modular framework.

Workload coverage and extensibility. Parallax’s concurrency models (TSPM and MPSPF) are mechanism-centric: they consume execution graphs and resource signatures (e.g., operator costs, bandwidth usage) rather than model-specific semantics. While we instantiate the interface using a DL-oriented profiler (MonoPerf), the interference models themselves are unchanged across domains. Extending Parallax to other GPU workloads (e.g., scientific computing) therefore primarily requires a compatible profiling front-end to extract kernel/operator traces.

Deep learning generality and LLMs. Our evaluation already covers representative LLM architectures, including encoder-style (BERT) and decoder-only autoregressive Transformers (GPT-2), which exercise the dominant operator motifs (Attention, MLP, LayerNorm). Very large LLMs (e.g., Qwen/Llama-class) often require multi-GPU parallelism due to memory footprint, which is beyond this article’s single-GPU sharing focus. Nonetheless, Parallax is scale-agnostic: as long as a workload fits within the co-execution memory budget and can be profiled, Parallax can predict its time-sliced/MPS co-execution performance independent of model size or name.

Integration and Feasibility. Parallax is designed as a high-fidelity performance oracle that can be queried by an external scheduler to score candidate placements and resource partitions. Its offline costs (TSPM calibration and MPS training) are one-time per model-hardware pair and can

be amortized over the deployment lifecycle, while online inference for the analytical model and XGBoost is microsecond-scale, enabling real-time evaluation of many placement options. To cope with runtime drift, Parallax naturally supports a *Predict–Schedule–Monitor–Refine* loop, where lightweight recalibration can be performed periodically using recent execution traces.

10 Conclusion

This article presents **Parallax**, a fine-grained performance prediction framework that systematically bridges the gap in modeling training–inference co-execution. By introducing tailored predictive models for the two dominant GPU concurrency mechanisms—time-slicing and MPS—Parallax enables precise characterization of resource contention and interaction effects. Extensive evaluations demonstrate that Parallax delivers highly accurate latency predictions across diverse co-location scenarios. Ultimately, by unifying analytical insights with data-driven modeling, Parallax provides a scalable and adaptive foundation for next-generation cluster schedulers to maximize GPU utilization while strictly safeguarding service-level objectives.

Acknowledgments

We would like to thank the anonymous reviewers for their insightful comments and suggestions.

References

- [1] alibaba. 2025. cluster-trace-gpu-2025. Retrieved from <https://github.com/alibaba/clusterdata/tree/master/cluster-trace-gpu-v2025>
- [2] Zhihao Bai, Zhen Zhang, Yibo Zhu, and Xin Jin. 2020. PipeSwitch: Fast pipelined context switching for deep learning applications. In *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation*. USENIX Association, 499–514. Retrieved from <https://www.usenix.org/conference/osdi20/presentation/bai>
- [3] Yoshua Bengio, Ian Goodfellow, Aaron Courville, et al. 2017. *Deep Learning*. MIT Press Cambridge, MA, USA.
- [4] Nicola Capodiec, Roberto Cavicchioli, Marko Bertogna, and Aingara Paramakuru. 2018. Deadline-based scheduling for GPU with preemption support. In *Proceedings of the 2018 IEEE Real-Time Systems Symposium*. IEEE, 119–130.
- [5] Tianqi Chen and Carlos Guestrin. 2016. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. Association for Computing Machinery, New York, NY, USA, 785–794.
- [6] Wenyan Chen, Chengzhi Lu, Huanle Xu, Kejiang Ye, and Chengzhong Xu. 2025. Multiplexing dynamic deep learning workloads with slo-awareness in gpu clusters. In *Proceedings of the 20th European Conference on Computer Systems*. 589–604.
- [7] Wenyan Chen, Zizhao Mo, Huanle Xu, Kejiang Ye, and Chengzhong Xu. 2023. Interference-aware multiplexing for deep learning in gpu clusters: A middleware approach. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–15.
- [8] Seungbeom Choi, Sunho Lee, Yeonjae Kim, Jongse Park, Youngjin Kwon, and Jaehyuk Huh. 2022. Serving heterogeneous machine learning models on multi-gpu servers with spatio-temporal sharing. In *Proceedings of the 2022 USENIX Annual Technical Conference*. 199–216.
- [9] Weihao Cui, Han Zhao, Quan Chen, Hao Wei, Zirui Li, Deze Zeng, Chao Li, and Minyi Guo. 2022. {DVABatch}: Diversity-aware {multi-entry}{multi-exit} batching for efficient processing of {dnn} services on {gpus}. In *Proceedings of the 2022 USENIX Annual Technical Conference*. 183–198.
- [10] Ruwen Fan, Tingxu Ren, Minhui Xie, Shiwei Gao, Jiwu Shu, and Youyou Lu. 2025. {GPREEPT}{GPU} Preemptive scheduling made general and efficient. In *Proceedings of the 2025 USENIX Annual Technical Conference*. 263–272.
- [11] X. Yu Geoffrey, Yubo Gao, Pavel Golikov, and Gennady Pekhimenko. 2021. Habitat: A runtime-based computational performance predictor for deep neural network training. In *Proceedings of the 2021 USENIX Annual Technical Conference*. 503–521.
- [12] google. 2025. AI Hypercomputer. Retrieved from <https://cloud.google.com/ai-hypercomputer/docs/overview>
- [13] Arpan Gujarati, Reza Karimi, Safya Alzayat, Wei Hao, Antoine Kaufmann, Ymir Vigfusson, and Jonathan Mace. 2020. Serving {DNNs} like clockwork: Performance predictability from the bottom up. In *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation*. 443–462.
- [14] Ziyi Han, Ruiting Zhou, Chengzhong Xu, Yifan Zeng, and Renli Zhang. 2024. Inss: An intelligent scheduling orchestrator for multi-gpu inference with spatio-temporal sharing. *IEEE Transactions on Parallel and Distributed Systems* (2024).

- [15] Cheol-Ho Hong, Ivor Spence, and Dimitrios S Nikolopoulos. 2017. GPU virtualization and scheduling methods: A comprehensive survey. *ACM Computing Surveys* 50, 3 (2017), 1–37.
- [16] Qinghao Hu, Peng Sun, Shengen Yan, Yonggang Wen, and Tianwei Zhang. 2021. Characterization and prediction of deep learning workloads in large-scale gpu datacenters. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–15.
- [17] Saksham Jain, Iljoo Baek, Shige Wang, and Ragunathan Rajkumar. 2019. Fractional GPUs: Software-based compute and memory bandwidth reservation for GPUs. In *Proceedings of the 2019 IEEE Real-Time and Embedded Technology and Applications Symposium*. IEEE, 29–41.
- [18] Myeongjae Jeon, Shivaram Venkataraman, Amar Phanishayee, Junjie Qian, Wencong Xiao, and Fan Yang. 2019. Analysis of {large-scale}{multi-tenant}{gpu} clusters for {dnn} training workloads. In *Proceedings of the 2019 USENIX Annual Technical Conference*. 947–960.
- [19] Xin Jin, Zhen Zhang, Yunshan Jia, Yun Ma, and Xuanzhe Liu. 2024. SDCC: Software-defined collective communication for distributed training. *Science China Information Sciences* 67, 9 (2024), 192104.
- [20] Alind Khare, Dhruv Garg, Sukrit Kalra, Snigdha Grandhi, Ion Stoica, and Alexey Tumanov. 2025. {SuperServe}:{Fine-grained} inference serving for unpredictable workloads. In *Proceedings of the 22nd USENIX Symposium on Networked Systems Design and Implementation*. 739–758.
- [21] Kevin Klues, Kyrlo Perelygin, and Pramod Ramarao. 2022. Improving GPU Utilization in Kubernetes. NVIDIA Technical Blog. Retrieved December 18, 2025 from <https://developer.nvidia.com/blog/improving-gpu-utilization-in-kubernetes/>
- [22] Azure Machine Learning. 2025. Retrieved from <https://azure.microsoft.com/en-us/products/machine-learning>
- [23] Seonho Lee, Amar Phanishayee, and Divya Mahajan. 2025. Forecasting GPU performance for deep learning training and inference. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. 493–508.
- [24] Jiamin Li, Hong Xu, Yibo Zhu, Zherui Liu, Chuanxiong Guo, and Cong Wang. 2023. Lyra: Elastic scheduling for deep learning clusters. In *Proceedings of the 18th European Conference on Computer Systems*. 835–850.
- [25] Ying Li, Yifan Sun, and Adwait Jog. 2023. Path forward beyond simulators: Fast and accurate gpu execution time prediction for dnn workloads. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*. 380–394.
- [26] Zhuohan Li, Lianmin Zheng, Yinmin Zhong, Vincent Liu, Ying Sheng, Xin Jin, Yanping Huang, Zhifeng Chen, Hao Zhang, Joseph E Gonzalez, et al. 2023. {AlpaServe}: Statistical multiplexing with model parallelism for deep learning serving. In *Proceedings of the 17th USENIX Symposium on Operating Systems Design and Implementation*. 663–679.
- [27] Jaiaid Mobin, Avinash Maurya, and M. Mustafa Rafique. 2023. Colti: Towards concurrent and co-located dnn training and inference. In *Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing*. 309–310.
- [28] Diksha Moolchandani, Anshul Kumar, and Smruti R. Sarangi. 2022. Performance and power prediction for concurrent execution on gpus. *ACM Transactions on Architecture and Code Optimization* 19, 3 (2022), 1–27.
- [29] Kelvin KW. Ng, Henri Maxime Demoulin, and Vincent Liu. 2023. Paella: Low-latency model serving with software-defined gpu scheduling. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 595–610.
- [30] NVIDIA. 2025. CUDA Multi-Process Service. Retrieved from https://docs.nvidia.com/deploy/pdf/CUDA_Multi_Process_Service_Overview.pdf
- [31] NVIDIA. 2025. Nvidia multi-instance GPU (MIG). Retrieved from <https://www.nvidia.com/en-us/technologies/multi-instance-gpu/>
- [32] NVIDIA. 2025. NVIDIA Nsight compute. Retrieved from <https://docs.nvidia.com/nsight-compute/NsightCompute/index.html>
- [33] NVIDIA. 2025. NVIDIA Nsight system. Retrieved from <https://docs.nvidia.com/nsight-systems/InstallationGuide/index.html>
- [34] NVIDIA. 2025. Time-Slicing GPUs in Kubernetes. Retrieved from <https://docs.nvidia.com/datacenter/cloud-native/gpu-operator/latest/gpu-sharing.html>
- [35] Ignacio Sañudo Olmedo, Nicola Capodieci, Jorge Luis Martinez, Andrea Marongiu, and Marko Bertogna. 2020. Dissecting the CUDA scheduling hierarchy: A performance and predictability perspective. In *Proceedings of the 2020 IEEE Real-Time and Embedded Technology and Applications Symposium*. IEEE, 213–225.
- [36] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. Pytorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems* 32 (2019).
- [37] Yifan Qiao, Shu Anzai, Shan Yu, Haoran Ma, Yang Wang, Miryung Kim, and Harry Xu. 2024. Conserve: Harvesting gpus for low-latency and high-throughput large language model serving. arXiv:2410.01228. Retrieved from <https://arxiv.org/abs/2410.01228>
- [38] Amazon SageMaker. 2025. Retrieved from <https://aws.amazon.com/sagemaker>

- [39] Sudipta Saha Shubha, Haiying Shen, and Anand Iyer. 2024. {USHER}: Holistic interference avoidance for resource optimized {ML} inference. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation*. 947–964.
- [40] Foteini Strati, Xianzhe Ma, and Ana Klimovic. 2024. Orion: Interference-aware, fine-grained gpu sharing for ml applications. In *Proceedings of the 19th European Conference on Computer Systems*. 1075–1092.
- [41] Jiali Wang, Yankui Wang, Mingcong Han, and Rong Chen. 2025. Colocating {ml} inference and training with fast {gpu} memory handover. In *Proceedings of the 2025 USENIX Annual Technical Conference*. 1657–1675.
- [42] Yidi Wang, Cong Liu, Daniel Wong, and Hyoseung Kim. 2024. GCAPS: GPU context-aware preemptive priority-based scheduling for real-time tasks. arXiv:2406.05221. Retrieved from <https://arxiv.org/abs/2406.05221>
- [43] Qizhen Weng, Wencong Xiao, Yinghao Yu, Wei Wang, Cheng Wang, Jian He, Yong Li, Liping Zhang, Wei Lin, and Yu Ding. 2022. {MLaaS} in the wild: Workload analysis and scheduling in {Large-Scale} heterogeneous {GPU} clusters. In *Proceedings of the 19th USENIX Symposium on Networked Systems Design and Implementation*. 945–960.
- [44] Bingyang Wu, Zili Zhang, Zhihao Bai, Xuanzhe Liu, and Xin Jin. 2023. Transparent {GPU} sharing in container clouds for deep learning workloads. In *Proceedings of the 20th USENIX Symposium on Networked Systems Design and Implementation*. 69–85.
- [45] Fei Xu, Jianian Xu, Jiabin Chen, Li Chen, Ruitao Shang, Zhi Zhou, and Fangming Liu. 2022. igniter: Interference-aware gpu resource provisioning for predictable dnn inference in the cloud. *IEEE Transactions on Parallel and Distributed Systems* 34, 3 (2022), 812–827.
- [46] Zhisheng Ye, Wei Gao, Qinghao Hu, Peng Sun, Xiaolin Wang, Yingwei Luo, Tianwei Zhang, and Yonggang Wen. 2024. Deep learning workload scheduling in gpu datacenters: A survey. *Computing Surveys* 56, 6 (2024), 1–38.
- [47] Hong Zhang, Yupeng Tang, Anurag Khandelwal, and Ion Stoica. 2023. {SHEPHERD}: Serving {DNNs} in the wild. In *Proceedings of the 20th USENIX Symposium on Networked Systems Design and Implementation*. 787–808.
- [48] Jeff Zhang, Sameh Elnikety, Shuayb Zarar, Atul Gupta, and Siddharth Garg. 2020. {Model-switching}: Dealing with fluctuating workloads in {Machine-Learning-as-a-Service} systems. In *Proceedings of the 12th USENIX Workshop on Hot Topics in Cloud Computing*.
- [49] Wei Zhang, Quan Chen, Ningxin Zheng, Weihao Cui, Kaihua Fu, and Minyi Guo. 2021. Toward qos-awareness and improved utilization of spatial multitasking gpus. *IEEE Transactions on Computers* 71, 4 (2021), 866–879.

Received 19 December 2025; revised 13 March 2026; accepted 17 June 2026